

Praktikum

Thema: Automated Valet Parking

Zur Lehrveranstaltung Autonomes Fahren

Inhalt

1	Einleitung	1
2	Strukturierung des Simulinkmodells	2
3	Praktikumsaufgaben	7
3.1	<i>Mindestanforderungen für das Bestehen des Praktikums</i>	7
3.2	<i>Zusatzpunkte für Modulprüfung.....</i>	8
3.3	<i>Meilensteine</i>	8
3.4	<i>Aufgaben in MATLAB/Simulink</i>	9
3.5	<i>Aufbauende Aufgaben in MATLAB/Simulink</i>	14
3.6	<i>Fragenteil</i>	16
4	Benötigte Blöcke aus der Simulinkbibliothek.....	17
5	Beispiele für die Arbeit mit MATLAB/Simulink	20
5.1	<i>Beispiel für den Aufruf eines Matlab Scripts in einem Matlab Function Block</i>	20
5.2	<i>Bearbeitung der Ein- und Ausgangsvariablen von einem Matlab Function-Block - Property Inspector</i>	21
5.3	<i>Mask</i>	23
6	Literaturverzeichnis.....	25

1 Einleitung

Im Rahmen dieses Praktikums wird das Automated Valet Parking (AVP) näher betrachtet. Dieses Praktikum soll ein grundlegendes Verständnis für die Realisierung solcher Fahrfunktionen in MATLAB/Simulink bieten. Beim Automated Valet Parking ist es möglich ein Fahrzeug in einer Drop-off Zone eines Parkplatzes abzuliefern. Das Fahrzeug sucht sich bei Aktivierung des AVPs selbständig ein Parkplatz und parkt dort ein. Eine weitere Funktion ermöglicht es das Fahrzeug zu rufen, wodurch es selbständig ausparkt und zu einer Pick-up Zone fährt. Dort kann es dann von dem Besitzer wieder entgegengenommen werden [1].

Ein zentraler Bestandteil des AVPs und anderer automatisierter Fahrfunktionen ist die Pfadplanung. Daher ist ein Verständnis dieses Prozesses eine wichtige Kompetenz. Im Rahmen des Praktikums wird die Pfadplanung in vereinfachter Form für ein Automated Valet Parking umgesetzt. Dabei wird der RRT*-Algorithmus als beispielhafte Methode vorgestellt und der Umgang mit MATLAB/Simulink eingeführt. Der RRT-Algorithmus (Rapidly exploring Random Tree) ist ein Verfahren zur Wegplanung. Im Gegensatz zu anderen Methoden konzentriert sich RRT auf zufällig ausgewählte Punkte. Bei der Erstellung eines Pfades vom Startpunkt zum Ziel wählt der Algorithmus zufällige Punkte aus und verzweigt sich in diese Richtung. Dabei vermeidet der Algorithmus bereits erkundete Knoten erneut zu besuchen [2].

MATLAB/Simulink ist eine weit verbreitete Entwicklungsumgebung, die sich besonders für die Simulation vielfältiger Problemstellungen eignet. Aufgrund ihrer Vielseitigkeit und der starken Präsenz in der Industrie sind Grundkenntnisse im Umgang mit dieser Software eine wertvolle Kompetenz.

Durch das Praktikum sollen folgende Lernziele erreicht werden:

- Automated Valet Parking kennenlernen und verstehen
- Trajektorienplanung verstehen
- Algorithmen für die Pfadplanung kennenlernen und anwenden
- Verständnis der grundlegenden Funktionsweise von MATLAB/Simulink
- Kennenlernen der wichtigsten Funktionsblöcke in Simulink
- Auswertung und Analyse von Simulationsdaten
- Visualisierung von Daten durch Plots
- Aufbau und Durchführung einer funktionierenden Simulation in Simulink
- Skriptübergreifende Nutzung von *Functions*

Als Grundlage für die Bearbeitung der Praktikumsaufgaben bieten sich die von Mathworks angebotenen Onramp Kurse für Matlab und Simulink. Diese finden Sie unter folgendem Link: https://matlabacademy.mathworks.com/de/?page=1&sort=featured&s_tid=getstart_mlacad

2 Strukturierung des Simulinkmodells

Dieses Kapitel soll Ihnen alle Bestandteile des Simulinkmodells vorstellen, sodass Sie die Funktionsweise verstehen und wissen welche Komponenten eingesetzt werden.

Funktionsweise des AVP-Simulinkmodells

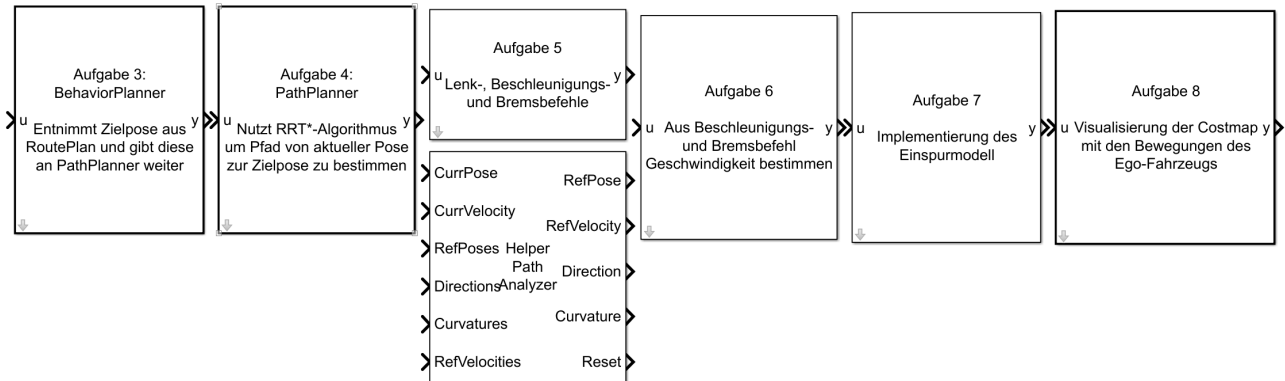


Abbildung 1 Gegebene Blöcke und Einteilung des Simulinkmodells in die Aufgabenbereiche

Die AVP-Simulation basiert auf mehreren Bestandteilen. Zu diesen Bestandteilen gehören Eingabedaten, Matlab Skripte und das Simulinkmodell. Tabelle 1 umfasst alle bereitgestellten Bestandteile.

Zu den Eingabedaten gehören der *RoutePlan*, der Informationen wie die Ausgangspose, die Zielpose und eine Reihe von Etappenzielen umfasst. Weiterhin enthält der *RoutePlan* Attribute wie eine maximale Geschwindigkeit und Informationen, die zur Einstellung des RRT-Algorithmus dienen. Zu den Eingabedaten zählen auch die .bmp-Dateien die den Parkplatz abbilden. Aus diesen Dateien wird die *Costmap* erstellt. Eine *Costmap* ist eine 2D-Karte. Die Elemente der Karte werden durch Zahlenwerte repräsentiert. Je höher die Werte sind, desto größer ist das Risiko.

Die Matlab-Skripte haben zum einen die Aufgabe die Eingabedaten zusammenzuführen und dem Simulinkmodell bereitzustellen und zum anderen gewisse Prozesse aus dem Simulinkmodell auszulagern und für eine bessere Strukturierung zu sorgen.

Ein Beispiel für den ersten Anwendungsfall ist das Matlab-Skript „Initialisierung“. Dieses Skript soll unter anderem den *RoutePlan* in den Matlab-Workspace laden, die *Costmap* durch das jeweilige ausgelagerte Skript erstellen lassen und in den Workspace laden. Außerdem soll es das Simulinkmodell öffnen und aktualisieren. In dem Matlab-Workspace werden alle Variablen angezeigt und gespeichert, die in Matlab erstellt oder geladen wurden. Weiterhin wird dort auch angezeigt, um welchen Datentyp es sich handelt. Diese Anzeige beschränkt sich jedoch nur auf die Arbeit in Matlab-Skripten. Variablen aus Simulink oder Variablen aus von Simulink aufgerufenen Matlab-Skripten werden nicht angezeigt. Dies kann aber durch den Simulink-Block „To Workspace“ eingestellt werden.

Ein Beispiel für den zweiten Anwendungsfall ist das Script „HelperPathAnalyzer“. Dieses Skript berechnet die Referenzposition und die Referenzgeschwindigkeit auf der Grundlage der aktuellen Position des Fahrzeugs.

Mit den Eingabedaten und den Matlab-Skripten kann das Simulinkmodell arbeiten. Abbildung 1 zeigt in welche Abschnitte das Modell unterteilt werden kann.

Der *BehaviorPlanner* hat die Aufgabe abhängig von der aktuellen Fahrzeugpose die nächste Zielpose aus dem *RoutePlan* zu entnehmen und dem *PathPlanner* zur Verfügung zu stellen. Weiterhin soll der *BehaviorPlanner* auch die Geschwindigkeiten übergeben und die Einstellung für den RRT*-Algorithmus ausgeben. Der Block *GoalChecker* soll durchgehend die Entfernung von der aktuellen Fahrzeugpose zu der Zielpose berechnen. Wenn die Entfernung einen festgelegten Wert unterschreitet, soll der Block ein Signal zu dem *BehaviorPlanner* übermitteln, sodass die nächste Zielpose ausgegeben wird.

Der *PathPlanner* ermittelt über den RRT*-Algorithmus eine Route zu der vom *BehaviorPlanner* ausgegebenen Zielpose. Dabei soll der RRT*-Algorithmus mit den vom *BehaviorPlanner* ausgegebenen Einstellung arbeiten.

Ziel des nächsten Modellabschnitts ist die Generierung von Steuerbefehlen. Dazu wird vom Path Smoother Spline die Routenführung vom *PathPlanner* geglättet, vom Velocity Profiler passende Geschwindigkeiten ausgegeben und vom *Helper Path Analyzer* die tatsächliche Fahrzeugpose mit der Referenzpose abgeglichen. Mit diesem Vorgehen können anschließend die Blöcke *Lateral Controller Stanley* und *Longitudinal Controller Stanley* Steuerbefehle bestimmen. Anschließend werden die Beschleunigungs- und Bremsbefehle über eine Integratorschaltung in eine Geschwindigkeit umgewandelt.

Der Lenkbefehl und die Geschwindigkeit bilden dann die Eingangssignale für das Einspurmodell. Mit den Ausgangssignalen vom Einspurmodell sind alle Werte für die Simulation vorhanden. Damit ist es möglich ein Plot mit der Fahrzeugbewegung auf der *Costmap* zu erstellen.

Benötigte Komponenten für die Bearbeitung der Praktikumsaufgaben

Nutzen Sie für die Bearbeitung die Dateien in dem bereitgestellten Ordner. In dem Ordner finden Sie die bmp-Dateien für die *Costmap*, eine m-Datei für die Erstellung der *Costmap*, eine weitere m-Datei für ein Initialisierungsskript, eine slx-Datei für die Erstellung des AVP-Simulinkmodells und eine Reihe von Helper-Dateien, die die Erstellung des Simulinkmodells vereinfachen sollen. Die Helper-Dateien müssen nicht bearbeitet werden.

Abbildung 2 soll die Zuständigkeiten der einzelnen Dateien veranschaulichen. Die blau markierten Dateien sind zu bearbeiten. In Tabelle 1 sind zusätzlich alle Dateien kurz beschrieben. In den zu bearbeitenden Dateien finden Sie Kommentare, die Ihnen als Leitfaden dienen und die Bearbeitung vereinfachen sollen.

Für die Bearbeitung der Aufgaben benötigen Sie die Automated Driving Toolbox. Diese ist in Simulink über Apps > Get Add-Ons installierbar.

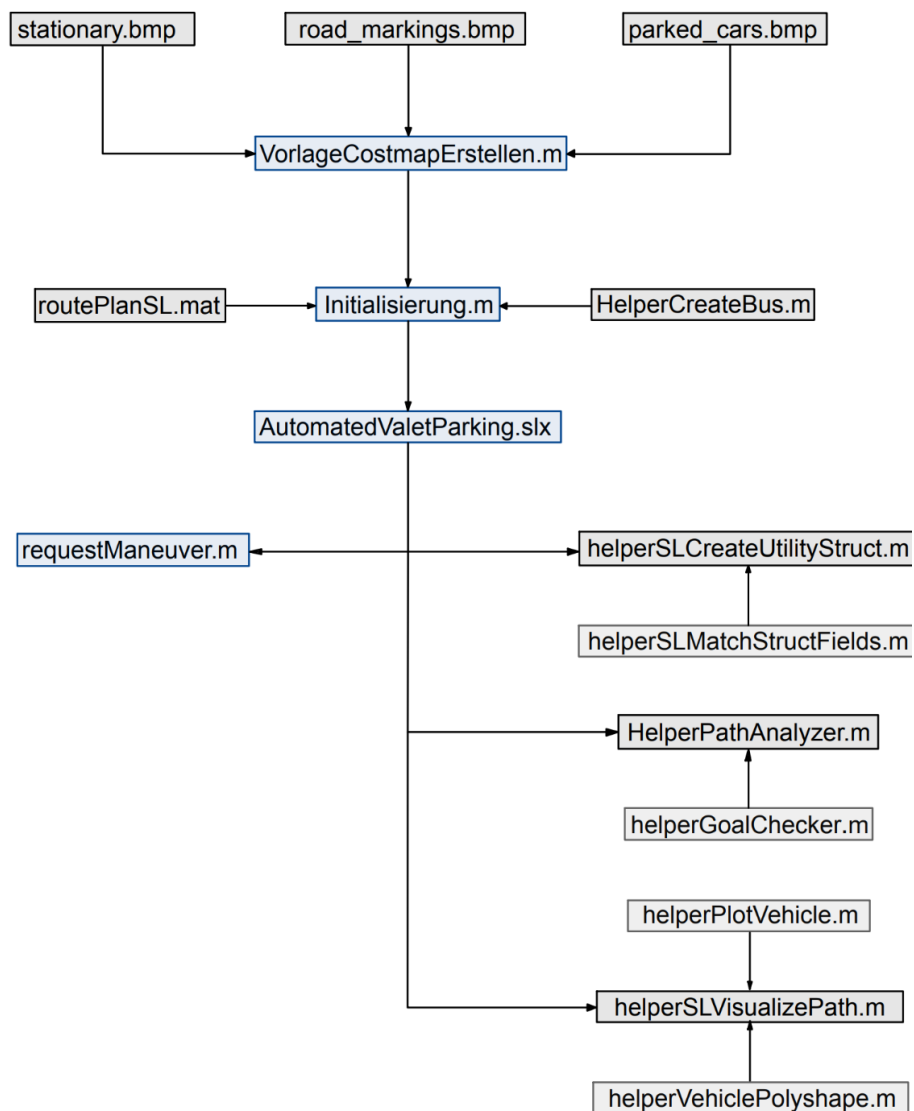


Abbildung 2 Übersicht zu den bereitgestellten Dateien und deren Anwendungsfeld.

Die blau markierten Dateien müssen eigenständig bearbeitet werden. Alle anderen bleiben unverändert.

Tabelle 1 Beschreibung der bereitgestellten Dateien (fettgedruckte Dateien sind für die Aufgabenstellungen zu bearbeiten)

Datei	Beschreibung
AutomatedValetParking.slx	Vorlage für Simulinkmodell: Die Vorlage enthält zwei Matlab Function Blöcke und den Block „HelperPathAnalyzer“. Ein Function Block dient als Vorlage für den Behaviour Planer. Der zweite Matlab Function Block dient als Vorlage für den Path Planner.
helperGoalChecker.m	Dieses Skript überprüft, ob die aktuelle Pose des Fahrzeugs die Zielpose aus dem RoutePlan erreicht hat.
HelperPathAnalyzer.m	Bestimmt die Referenzsignale, die der Vehicle Controller (Lateral- und Longitudinal Controller Stanley) benötigt, um die Lenkung und die Beschleunigung des Fahrzeugs zu regeln.
helperPlotVehicle.m	Visualisiert das Ego-Fahrzeug im Plot an der aktuellen Pose (CurrentPose) mit den jeweiligen Maßen (vehicleDims), die durch helperSLCreateUtilityStruct.m bestimmt werden.
HelperCreateBus.m	Erstellt die Bus-Architekturen „plannerConfigBus“, „speedConfigBus“ und „vehicleInfoBus“. Das Skript legt fest, welche Variablen mit welchem Datentyp in der jeweiligen Bus-Architektur enthalten sein soll.
HelperSLCreateUtilityStruct.m	Konvertiert „vehicleDimensions“ die „vehicleCostmap“ in ein „struct arrays“, damit diese in Simulink nutzbar ist.
helperSLMatchStructFields.m	Übernimmt Werte aus den Objekten und kopiert diese in die Structs.
helperSLVisualizePath.m	Visualisiert auf der Costmap die geplante Route, die zurückgelegte Route und die aktuelle Fahrzeugpose.
helperVehiclePolyshape.m	Erstellt die Visualisierung des Fahrzeugs im Plot. Dafür nutzt das Skript die Fahrzeugdimensionen, die bei der Erstellung der Costmap festgelegt werden. Das Skript stellt zudem sicher, dass die Visualisierung durchgehend an der aktuellen Pose erscheint.
Initialisierung.m	Dieses Skript soll das Simulinkmodell öffnen und diesem die aktuelle Costmap und den aktuellen RoutePlan übergeben. Anschließend soll die Simulation durch das Skript gestartet werden.
requestManeuver.m	Dieses Skript dient zur Bestimmung der nächsten Zielpose, der Geschwindigkeitsvorgaben und der Einstellungen für den RRT-Algorithmus.

Datei	Beschreibung
routePlanSL.mat	Diese Datei enthält die geplante Route in Form einer Tabelle mit Start- und Zielpunkten. Weiterhin enthält diese Tabelle eine dritte Spalte mit Attributen, die beispielsweise eine maximale Geschwindigkeit festlegt.
VorlageCostmapErstellen.m	In diesem Skript sollen die .bmp-Dateien übereinandergelegt und zu einer Costmap umgewandelt werden. Die Costmap dient als Umgebung für die AVP-Simulation.
parked_cars.bmp	Die .bmp-Dateien enthalten jeweils eine Informationsebene, die zusammengefügt die Costmap bilden.
road_markings.bmp	
stationary.bmp	

3 Praktikumsaufgaben

In den folgenden Unterkapiteln ist beschrieben welche Mindestanforderungen bestehen, um das Praktikum zu bestehen, welche Aufgaben in MATLAB/Simulink zu bearbeiten sind und welche Fragen schriftlich beantwortet werden müssen.

3.1 Mindestanforderungen für das Bestehen des Praktikums

Zum Bestehen müssen die in Unterpunkt 3.1 gestellten Aufgaben vollständig und erfolgreich bearbeiten, sodass Sie den in Abbildung 10 gezeigten Plot mit einem sich bewegenden Fahrzeug und eingezeichnetem Pfad erhalten. Weiterhin müssen die in Unterpunkt 3.6 gestellten Fragen vollständig beantworten. Halten Sie dabei die in Tabelle 2 aufgelisteten Kriterien ein.

Tabelle 2 Formatierungsvorgaben

Formalia	Erläuterung
Inhalt	Min. 3 Seiten Text
Bezeichnung der pdf-Datei bei Abgabe	Name_Nachname_Matrikelnummer_Fragenteil
Papierformat	DIN A4
Seitenränder	Oben und unten 2 cm; links 2 cm; rechts 2 cm
Seitenzahlen	Zentriert am Seitenende Außer dem Titelblatt sind alle Seiten zu nummerieren. Die Seiten vor dem Textteil (falls vorhanden, z. B. Titelblatt, Inhaltsverzeichnis, Tabellen- und Abkürzungsverzeichnis) sollten mit römischen Großbuchstaben nummeriert werden (I, II, III, IV etc.), wobei die Seitenzählung auf der Seite I (Titelblatt) nicht erscheint. Die Seiten des Textteils werden mit arabischen Zahlen (1, 2, 3 etc.) nummeriert. Diese Seitenzahlen werden bis zum Schluss, also auch durch den Anhang hindurch (falls vorhanden), weitergeführt.
Schrifttyp	Allgemeiner Text – Arial 11 Pkt. - o.Ä.; Überschriften 13 bzw. 12 Pkt., Blocksatz
Zeilenabstand	1,5
Satz	Blocksatz und Silbentrennung

Formalia	Erläuterung
Fußnoten	Arial 10 Pkt. - o.Ä., Blocksatz
Absätze	Nach gedanklicher Gliederung – 6 Pkt. Abstand nach Zeilenumbruch
Zitierstandard	IEEE-Zitierstil, z.B. Lorem ipsum dolor sit amet, consetetur sadipscing elitr [1], sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua [2]. [1] J. Behrens, Einführung in die Fahrzeugtechnik, Canada: Autoverlag, 2022

3.2 Zusatzpunkte für Modulprüfung

Durch das Praktikum können Sie insgesamt 30 Punkte für die Modulprüfung erhalten. Diese Punkte teilen sich wie folgt auf:

Aufgaben 9 – 13 (aus Unterpunkt 3.5) → 15 Punkte

Fragenteil (Unterpunkt 3.6) → 15 Punkte

3.3 Meilensteine

MS 1 Frist: 29.05.2026, 23:55 Uhr

Abgabe der MATLAB/Simulink-Dateien, die Ihre Lösungen für die Aufgaben aus Unterpunkt 3.4 enthalten.

Benennung der zip-Datei: MS1_AF_Nachname_Name_Matrikelnummer

MS 2 Frist: 03.07.2026, 23:55 Uhr

Abgabe Ihrer Antworten auf die Fragen aus Unterpunkt 3.6 sowie der MATLAB/Simulink-Dateien zu den Zusatzaufgaben aus Unterpunkt 3.5.

Benennung der pdf-Datei: MS2_AF_Nachname_Name_Matrikelnummer

Benennung der zip-Datei: MS2_AF_Zusatz_Nachname_Name_Matrikelnummer

3.4 Aufgaben in MATLAB/Simulink

Aufgabe 1: Erstellen Sie einen Code, der aus den .bmp-Dateien eine *Costmap* generiert. Diese *Costmap* soll einen Parkplatz widerspiegeln, auf dem einige Parkplätze belegt und einige frei sind. Auf diesem Parkplatz soll später das Ego-Fahrzeug die gewünschte Route abfahren und auf dem gewünschten Parkplatz einparken. Nutzen Sie für die Lösung der Aufgabe die Datei „VorlageCostmapErstellen.m“.

Ziel dieser Aufgabe ist die Generierung eines Plots, der genauso aussieht wie Abbildung 3.

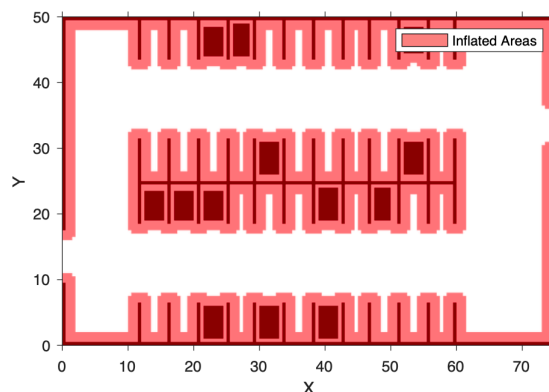


Abbildung 3 Plot der zu erstellenden Costmap mit eingezeichneten „inflated areas“

Aufgabe 2: Erstellen Sie mithilfe der Datei „Initialisierung.m“ ein Skript, mit dem Sie die Funktion zur Erstellung der Costmap aufrufen und diese sowie den *RoutePlan* in den Matlab Workspace laden. Weiterhin soll das Skript das Simulinkmodell (AutomatedValetParking.slx) öffnen und dem Modell die geladenen Daten bereitstellen.

Sobald alle Schritte richtig umgesetzt sind, sollte sich das Simulinkmodell automatisch öffnen und im Matlab Workspace die in gezeigten Werte ablesen lassen.

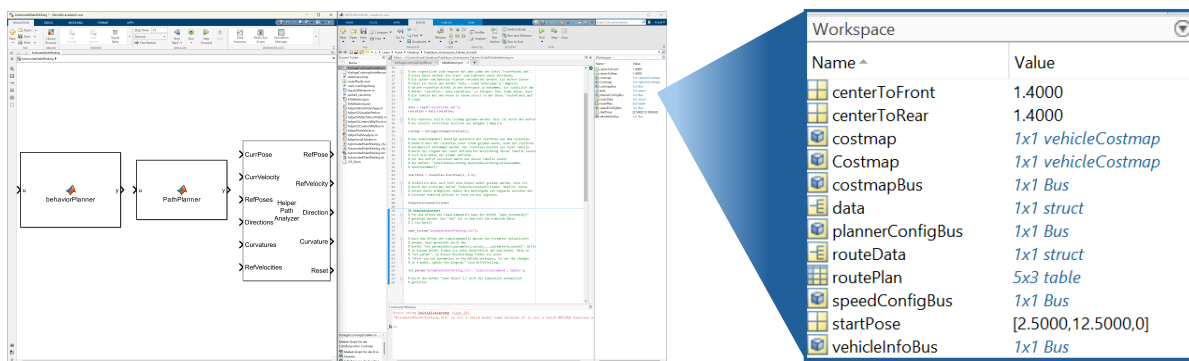


Abbildung 4 Fenster nach erfolgreicher Bearbeitung von Aufgabe 2

Aufgabe 3: Erstellen Sie einen Simulink Block, der aus dem RoutePlan (routePlanSL.mat) eine Pose entnimmt und als Ausgang ausgibt. Nutzen Sie dafür den „Matlab Function“-Block namens „BehaviorPlanner“ in der Datei „AutomatedValetParking.slx“. Für diese Aufgabe ist die Erstellung einer Mask für den Matlab Function Block notwendig. Die Vorgehensweise dafür ist unter Kapitel 5.3 beschrieben.

Die Hauptaufgabe dieses Blocks ist die Bereitstellung der Zielpose für den PathPlanner.

BehaviorPlanner: Eingang → routePlanStruct (über Mask), currentPose
 Ausgang → NextGoal

Als Ergebnis sollten Sie sich, wie in Abbildung 5 gezeigt, mithilfe des Display-Blocks die Zielpose anzeigen lassen können.

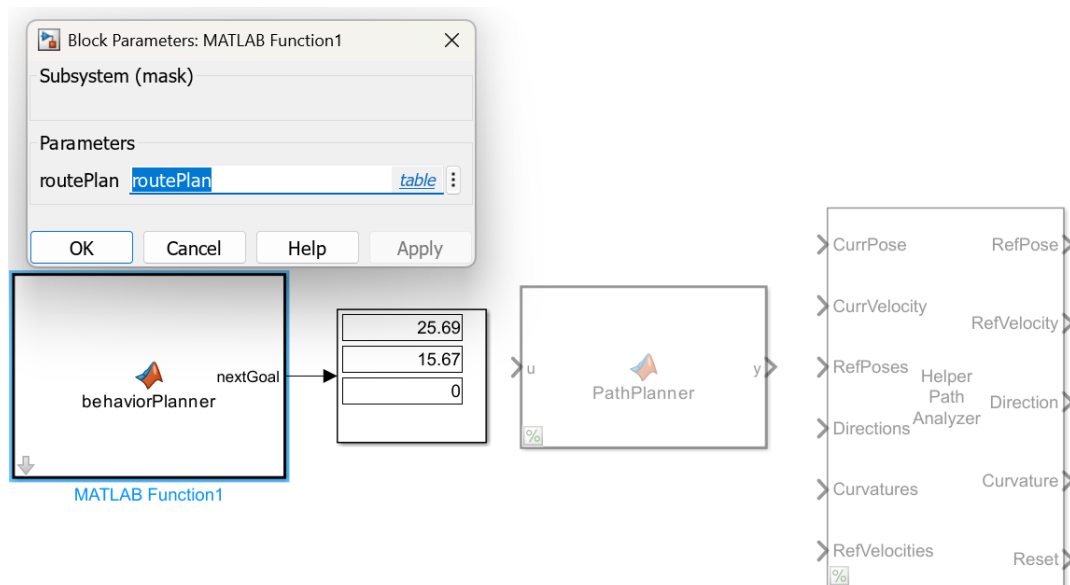


Abbildung 5 Simulinkmodell nach Bearbeitung von Aufgabe 3

Aufgabe 4: Implementieren Sie einen Block, der die vom BehaviorPlanner ausgegebene Zielpose aufgreift. Der Block soll dann von der aktuellen Pose, auf der sich das Fahrzeug gerade befindet, zur Zielpose eine Route berechnen. Dafür soll der RRT*-Algorithmus verwendet werden. Dies ist mit dem Befehl „pathPlannerRRT“ möglich. Der Block soll anschließend die Route in Form von Referenz-Posen und Fahrtrichtung ausgeben. Nutzen Sie dafür den Block namens „PathPlanner“, der schon in der slx-Datei vorhanden ist.

PathPlanner

Eingang: currentPose, NextGoal, costmapStruct und vehicleDimsStruct (über Mask ausgehend von Costmap)

Ausgang: RefPoses, Directions

Abbildung 4 zeigt das Simulinkmodell nach der Bearbeitung von Aufgabe 4. Da noch keine richtige Fahrzeugpose erstellt wurde, wird diese durch drei Constant-Blöcke und einem „Mux“ vorgetäuscht.

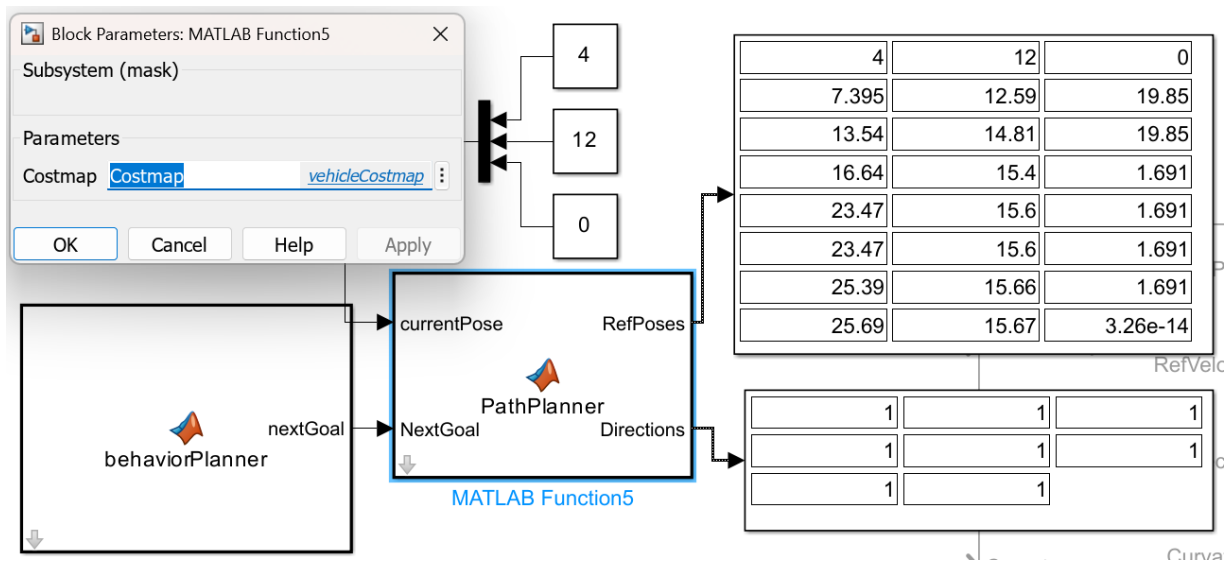


Abbildung 6 Simulinkmodell nach Bearbeitung von Aufgabe 4

Aufgabe 5: Erweitern Sie das Simulinkmodell so, dass aus den Referenz-Posen und der Fahrtrichtung Lenk-, Beschleunigungs- und Bremsbefehle generiert werden. Nutzen Sie dafür die Blöcke:

- Path Smoother Spline,
- Velocity Profiler,
- Helper Path Analyzer mit HelperPathAnalyzer.m,
- Lateral Controller Stanley und
- Longitudinal Controller Stanley

Bei dieser Anwendung muss im Block „Path Smoother Spline“ ein Häkchen bei „Show CumLengths and Curvatures output ports“ gesetzt werden. Im Velocity Profiler sollen die Werte wie folgt eingestellt werden:

- Maximum longitudinal acceleration (m/s^2): 2
- Maximum longitudinal deceleration (m/s^2): 4
- Maximum lateral acceleration (m/s^2): 2

Im Block Lateral Controller Stanley muss unter „Vehicle model“ das „Dynamic bicycle model“ ausgewählt werden. Weiterhin sind die Werte auf die VehicleDimensions anzupassen. In dieser Aufgabe müssen auch persistente Variablen im PathPlanner definiert werden. Dafür sind die Anweisungen in den Kommentaren zu befolgen.

Auf Abbildung 7 können Sie Richtwerte für die Lenk-, Beschleunigungs- und Bremsbefehle zu erwarten sind. Auch hier können Sie wieder noch nicht gebildete Signale durch Constant-Blöcke bilden. Für die aktuelle Fahrzeugpose können Sie ein Matlab Function-Block verwenden. Mit diesem können Sie über „Size“ im Property Inspector das Signalformat einstellen, das der Helper Path Analyzer erwartet.

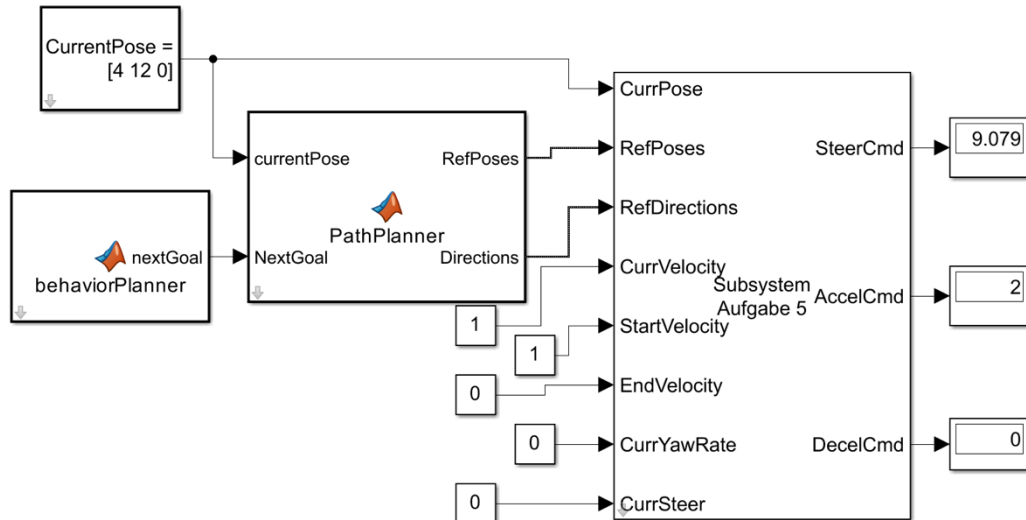


Abbildung 7 Simulinkmodell nach Aufgabe 5

Aufgabe 6: Implementieren Sie eine Integratorschaltung, die aus dem Beschleunigungs- und Bremsbefehl sowie der Fahrtrichtung eine Geschwindigkeit berechnen.

Als Ausgabedaten sollten Sie einen konstanten Lenkbefehl und eine stetig steigende Geschwindigkeit erhalten.

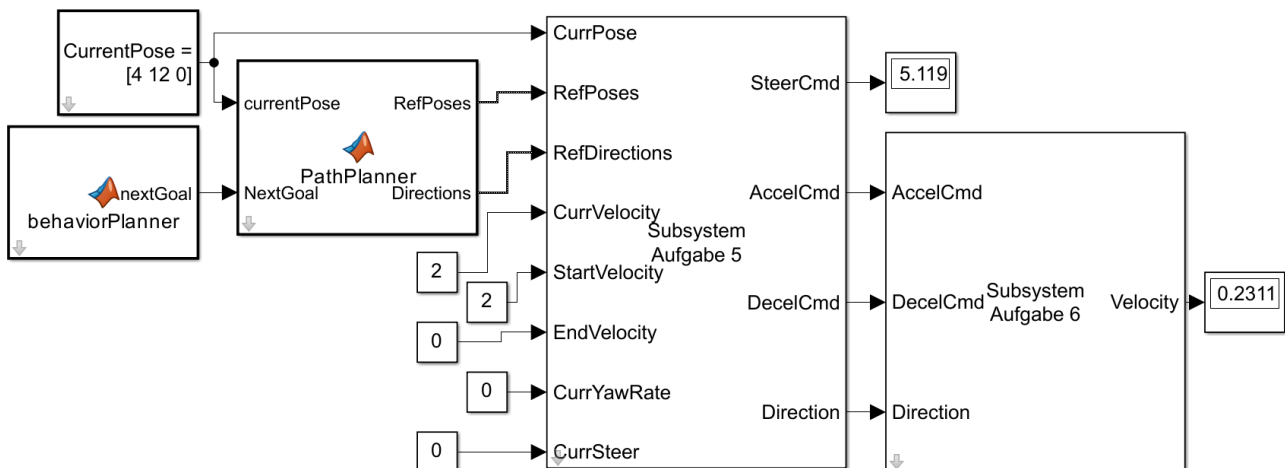


Abbildung 8 Simulinkmodell nach Aufgabe 6

Aufgabe 7: Implementieren Sie das Einspurmodell mit dem entsprechenden Simulink-Block. Führen Sie die entsprechenden, durch das Einspurmodell generierten, Signale an die jeweiligen Blöcke zurück (z. B. CurrentPose an Lateral Controller Stanley, Helper Path Analyzer, motionPlanning, BehaviourPlanner). Die Ausgänge für die laterale Geschwindigkeit und dem Gierwinkel werden nicht benötigt. Der Ausgang „Info“ gibt eine ganze Reihe von Werten aus. Über den Block „Bus Selector“ sind einzelne Werte auswählbar. Dafür muss die Simulation allerdings einmal gestartet werden. Über den Ausgang „Info“ ist die Generierung der aktuellen Fahrzeugpose unter Nutzung des Bus Selectors und dem Mux-Block möglich. Nutzen Sie „Unit Delay“-Blöcke um Algebraische Schleifen zu verhindern.

Bei der aktuellen Fahrzeugpose ist es auch möglich im „Unit Delay“-Block eine Ausgangsbedingung festzulegen. Hier ist die StartPose zu nutzen, die im Initialisierungsskript bestimmt wurde. Wenn diese Einstellung nicht vorgenommen wird, arbeitet der RRT*-Algorithmus zu Beginn mit der Pose [0 0 0]. Diese liegt in einem nicht befahrbaren Raum. Daher würde dies einen Fehlermeldung hervorrufen.

In den Einstellungen des Einspurmodells müssen die Werte „Initial inertial frame longitudinal position, X_o [m]:“ und „Initial inertial frame lateral displacement, Y_o [m]:“ parametrisiert werden. Auch dafür ist die Variable startPose aus der Initialisierung zu nutzen.

Nach erfolgreicher Bearbeitung sollte Ihr Modell eine vollständige Fahrzeugregelung enthalten und an der ersten Zielpose des RoutePlans anhalten. Abbildung 9 enthält Signalgrößen zur Orientierung.

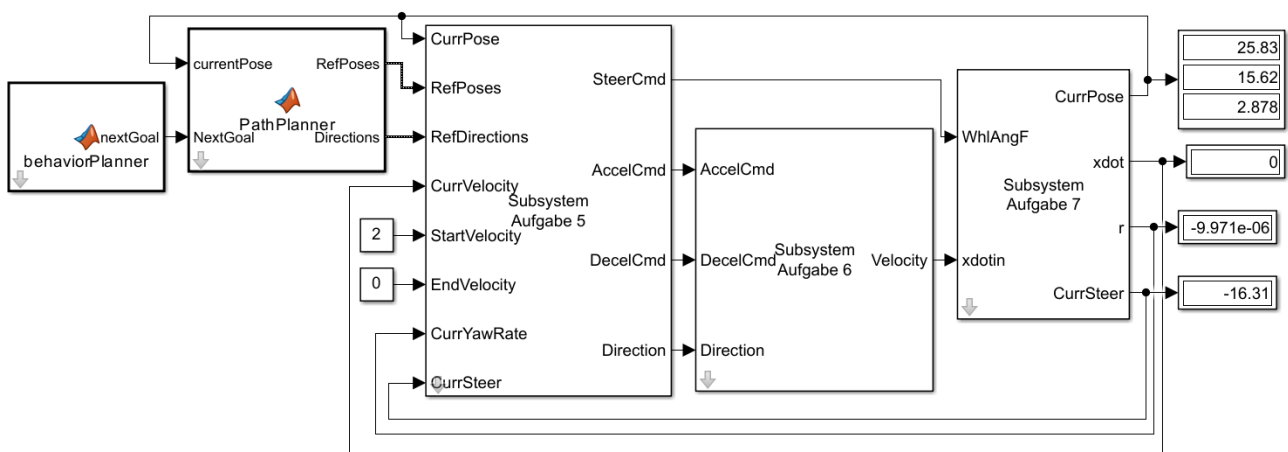


Abbildung 9 Simulinkmodell nach Aufgabe 7

Aufgabe 8: Erstellen Sie ein Matlab Function Block, der die Costmap plottet. Weiterhin soll in dem Plot das Fahrzeug, die berechnete Route und die tatsächlich zurückgelegte Route visualisiert werden. Nutzen Sie dafür die helper-Dateien "helperSLVisualizePath.m" und "helperPlotVehicle.m". Bei dieser Aufgabe muss die Function als extrinsic deklariert werden. Dies geschieht über den Befehl „coder.extrinsic(function)“.

Eine Deklaration als extrinsic wird angewendet, wenn statt einer Codegenerierung (Erzeugung von C/C++-Code), die Matlab-Engine zur Ausführung verwendet werden soll. Dies ist der Fall, wenn die Function nur Anzeige- oder Protokollierungsvorgänge durchführen soll und das Ergebnis der Simulation nicht beeinflusst.

Eingang: costmapStruct und vehicleDimsStruct (über Mask, ausgehend von Costmap), refPoses, currentPose, currentSteer

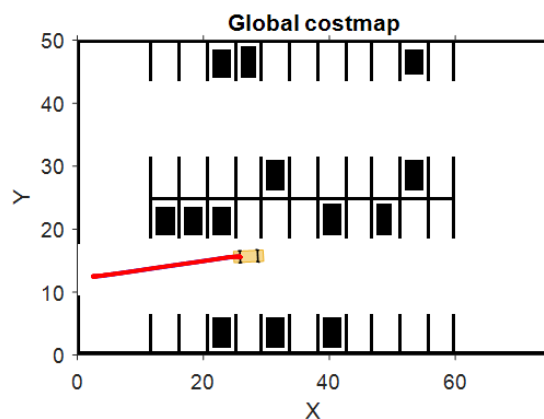


Abbildung 10 Plot von Costmap mit Fahrzeugbewegung

3.5 Aufbauende Aufgaben in MATLAB/Simulink

Aufgabe 9: In dieser Aufgabe soll das Modell so erweitert werden, dass auch die restlichen Zielposen aus dem RoutePlan angefahren werden.

Abbildung 11 zeigt die Route, die das Ego-Fahrzeug am Ende der Simulation zurückgelegt haben sollte.

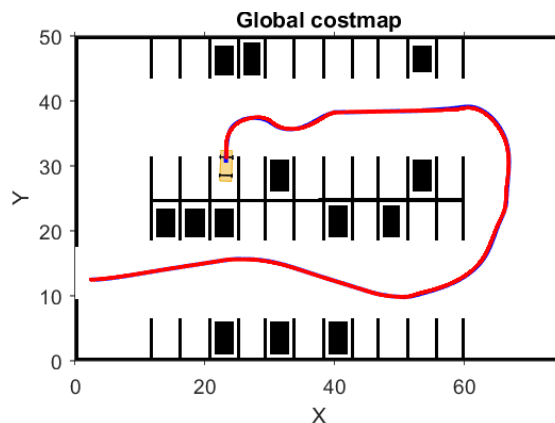


Abbildung 11 Plot mit vollständig befahrener Route

Aufgabe 10: Erweitern Sie das Modell so, dass nach dem Erreichen der letzten Zielpose die Simulation automatisch beendet wird. Nutzen Sie dafür den Block „Stop Simulation“.

Aufgabe 11: Nutzen Sie die Datei „requestManeuver.m“ um die Function zur Bestimmung der nächsten Zielpose in ein Matlab-Skript auszulagern. Rufen Sie die Function im BehaviorPlanner auf, damit weiterhin alle Zielposen angefahren werden.

Aufgabe 12: Implementieren Sie ein Bus-Signal, das die im RoutePlan festgelegten Geschwindigkeiten dem Velocity Profiler übergeben. Nutzen Sie dafür die helper-Datei „helperCreateBus.m“. Wenn alles korrekt eingestellt ist, sollte ein Scope-Block die in Abbildung 12 gezeigten Geschwindigkeitsverläufe ausgeben.

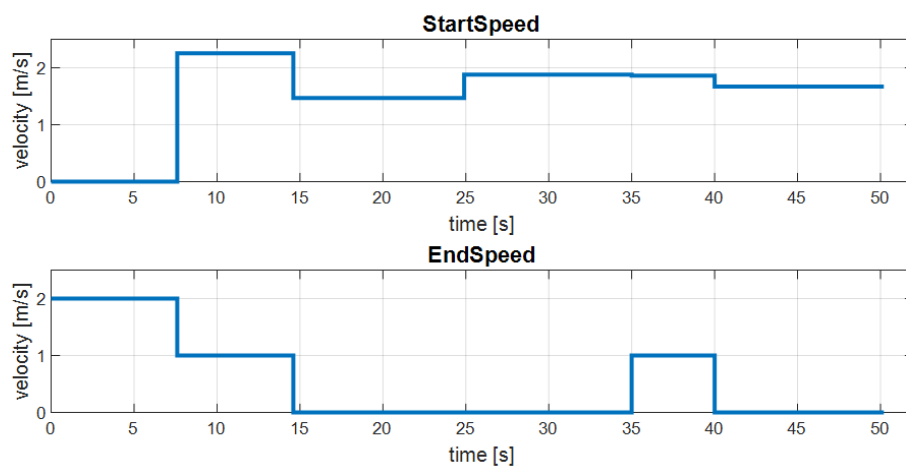


Abbildung 12 Geschwindigkeitseinstellungen für Velocity Profiler

Aufgabe 13: Erweitern Sie das Modell so, dass je nach Manöver der RRT*-Algorithmus unterschiedlich eingestellt wird. Erweitern Sie dafür das requestManeuver-Skript und geben Sie die Parameter durch ein Bus-Signal an den PathPlanner weiter. Nutzen Sie dafür den plannerConfigBus, der in der Datei „helperCreateBus.m“ definiert ist. Abbildung 13 enthält den Verlauf der einzelnen Parameter.

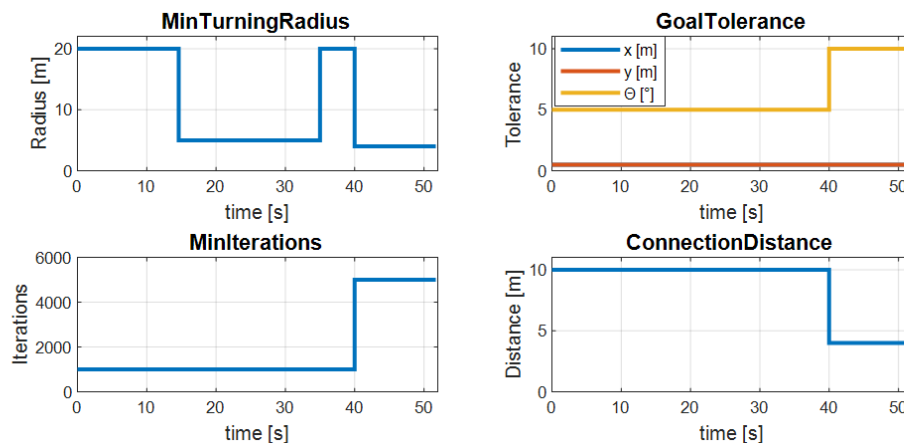


Abbildung 13 Verlauf der Parameter zur RRT*-Einstellung

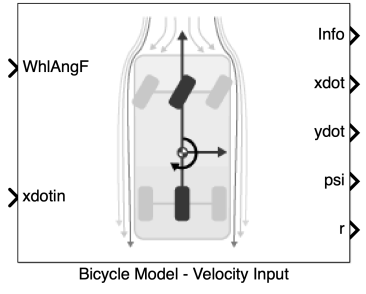
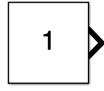
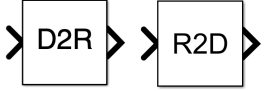
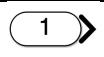
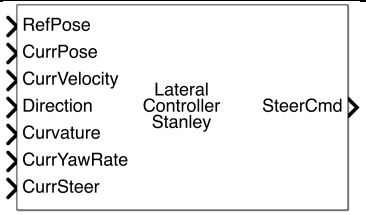
3.6 Fragenteil

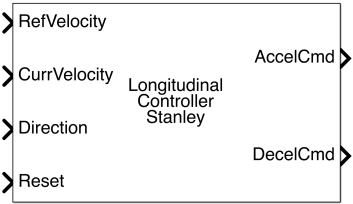
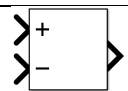
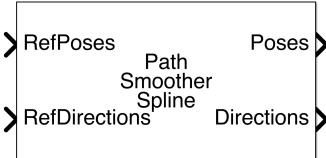
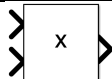
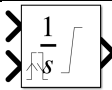
1. Beschreiben Sie die einzelnen Schritte, die das Simulinkmodell durchläuft und wie diese Schritte im Modell konkret umgesetzt sind.
2. Recherchieren Sie welchen AVP-Typen es gibt und ordnen Sie dem Simulink-Beispiel einen AVP-Typ zu. Begründen Sie Ihre Antwort und wodurch unterscheidet sich das AVP-System in dem hier behandelten Matlab/Simulink-Modell zu realen AVP-Systemen?
3. Womit muss ein AVP-fähiges Fahrzeug und ggf. die Infrastruktur ausgestattet sein?
4. Welche Höchstgeschwindigkeit und welcher Mindestabstand zu Hindernissen sind durch die Norm ISO 23374-1:2023 festgelegt? Die Norm finden Sie im Ilu-Kurs.
5. Beschreiben Sie das Schaubild unter "6.1.2 Relationship of operation function" aus der Norm ISO 23374-1:2023. Welche Schritte sind auf das Simulink Modell übertragbar?
6. Nennen Sie drei mögliche Testszenarien aus dem Katalog "Technischer Anforderungskatalog für die autonome Fahrfunktion „Automated Valet Parking (AVP)“" und was mit diesen getestet werden.
Den Anforderungskatalog finden Sie im Ilu-Kurs.
7. Welche Sicherheitsanforderungen stellt die ISO 23374-1:2023 an AVP-Systeme, die im Simulink-Modell nicht abgebildet sind?

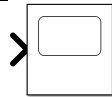
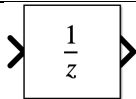
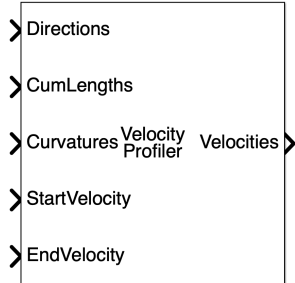
4 Benötigte Blöcke aus der Simulinkbibliothek

Neben den drei gegebenen Matlab Function-Blöcken und den Helper Path Analyzer wird zusätzlich noch eine Reihe anderer Blöcke benötigt. In der Tabelle 3 werden alle Blöcke aufgeführt und kurz erläutert, die für die Bearbeitung benötigt werden.

Tabelle 3 Blöcke aus der Simulinkbibliothek, die für die Bearbeitung der Aufgaben benötigt werden

Block-Bezeichnung	Funktion	Block	Benötigt für Aufgabe
Bicycle Model – Velocity Input	Implementiert ein starres zweiachsiges Einspurmodell zur Berechnung der Längs-, Quer- und Gierbewegung. Der Block berücksichtigt die Masse des Fahrzeugs, den Luftwiderstand und die Gewichtsverteilung. Zur Lösung dieser Aufgabe wird die Version mit der Geschwindigkeit als Eingabe.		7
Bus Creator	Kombiniert Eingangssignale zu einem Bus, der die separaten Identitäten der einzelnen Signale beibehält.		
Bus Selector	Extrahiert ausgewählte Signale aus dem Bus.		7, 12
Constant	Gibt eine Konstante aus.		
Degrees to Radians/ Radians to Degrees	Rechnet einen Wert mit der Einheit Grad in Bogenmaß um oder umgekehrt.		7
Display	Zeigt den Wert eines Signals während der Simulation.		
Inport	Verbindet ein externes System mit einem Subsystem.		
Lateral Controller Stanley	Bestimmt den Lenkbefehl (in Grad) für das Einspurmodell. Vergleicht die aktuelle Pose mit der RefPose und gleicht die Abweichung aus.		5

Block-Bezeichnung	Funktion	Block	Benötigt für Aufgabe
Longitudinal Controller Stanley	Bestimmt die Beschleunigungs- und Verzögerungsbefehle (in m/s), die das Einspurmodell später benötigt. Dazu vergleicht der Block die Referenzgeschwindigkeit und die aktuelle Geschwindigkeit.	 The diagram shows a block labeled 'Longitudinal Controller Stanley'. It has four inputs on the left: 'RefVelocity', 'CurrVelocity', 'Direction', and 'Reset'. It has two outputs on the right: 'AccelCmd' and 'DecelCmd'.	5
Matlab Function	Ermöglicht das Schreiben von Matlab-Funktionen in einem Simulinkmodell. Die Funktion während jeder Simulation durchgehend ausgeführt.	 The diagram shows a block labeled 'fcn' with a MATLAB logo. It has an input 'u' on the left and an output 'y' on the right.	3,4,6,7,8
Minus	Subtrahiert die Eingabewerte.	 The diagram shows a block with a '+' sign on top and a '-' sign on the bottom. It has two inputs on the left and one output on the right.	6
Mux	Kombiniert Eingaben mit demselben Datentyp und derselben Komplexität zu einem virtuellen Vektor. Ermöglicht die Gruppierung von Signalen.	 The diagram shows a block with multiple input lines on the left and one output line on the right, representing a multiplexer.	7
Outport	Dient als Ausgang eines Subsystems. Kann beispielsweise ein Signal aus einem Subsystem wieder in das übergeordnete Simulinkmodell befördern.	 The diagram shows a block with a '1' inside, representing an outport.	
Path Smoother Spline	Glättet den für das Fahrzeug generierten Pfad, damit dieser besser befahrbar ist.	 The diagram shows a block labeled 'Path Smoother Spline'. It has two inputs on the left: 'RefPoses' and 'RefDirections'. It has two outputs on the right: 'Poses' and 'Directions'.	5
Product	Multipliziert die Eingabewerte miteinander.	 The diagram shows a block with an 'x' inside, representing a product block.	6
Integrator	Integriert die Eingabewerte in Bezug auf Zeit. Hier wird ein Integrator mit der Einstellung „External reset: either“ → wird zurückgesetzt, wenn Reset-Signal von 0 auf 1 oder umgekehrt wechselt Grenzen: 0 – inf	 The diagram shows a block with an integral symbol and a '1/s' label, representing an integrator.	6

Block- Bezeichnung	Funktion	Block	Benötigt für Aufgabe
Scope	Zeigt den Verlauf der Eingabewerte über die Zeit an.		
Stop Simulation	Wenn der Eingabewert true ist, wird die Simulation beendet.		10
Subsystem	Öffnet eine neue Ebene des Simulinkmodells. Dient als Hilfe zur Ordnung des Modells.		
Unit Delay	Hält Eingangswert zurück und verzögert diese um eine Abtastperiode.		7
Velocity Profiler	Generiert eine Geschwindigkeitsabfolge, die eine festgelegte Maximalgeschwindigkeit, Längsbeschleunigung, Querb beschleunigung und ein Längsruck nicht überschreiten.		5

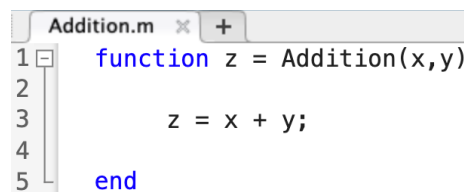
5 Beispiele für die Arbeit mit MATLAB/Simulink

In diesem Abschnitt sollen einige grundlegende Funktionen erläutert werden.

5.1 Beispiel für den Aufruf eines Matlab Scripts in einem Matlab Function Block

In Aufgabe 3 muss durch den Matlab Function Block „behaviorPlanner“ das Matlab Script „requestManeuver.m“ aufgerufen werden. Für ein besseres Verständnis soll dieses Beispiel das nötige Vorgehen nochmal erläutern.

Dazu dient das in Abbildung 14 gezeigte Matlab Script, in dem zwei Variablen miteinander addiert werden. Wichtig ist dabei, dass die Dateibenennung („Addition.m“) mit dem Namen der Function („Addition(x, y)“) übereinstimmt.



```
1 function z = Addition(x,y)
2
3     z = x + y;
4
5 end
```

Abbildung 14 Matlab Script mit Additionsaufgabe

Functions werden wie folgt definiert:

function [<Ausgangswerte>] = Functionname(<Eingangswerte>)

In Simulink sollen dann zwei Constant-Blöcke die Eingangswerte für den Matlab Function-Block liefern. Der Aufbau ist in Abbildung 15 dargestellt.

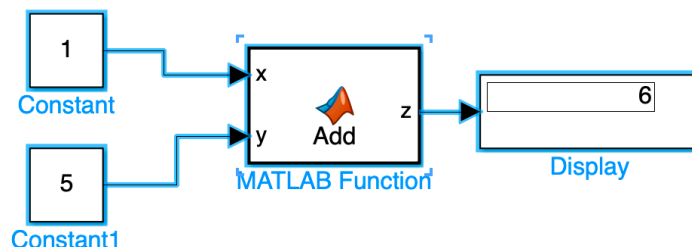
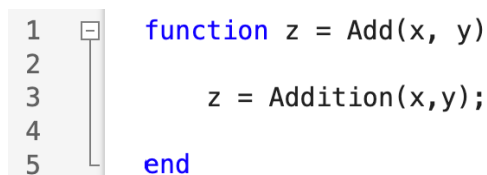


Abbildung 15 Simulinkmodell für die Durchführung der Addition

Die Ein- und Ausgänge des Blockes werden auf dieselbe Weise wie auch die Eingangswerte der zuvor erstellten Function definiert. Den Code in dem Matlab Function-Block zeigt Abbildung 16. Durch den einfachen Aufruf der zuvor definierten Function, wird diese ausgeführt.



```
1 function z = Add(x, y)
2
3     z = Addition(x,y);
4
5 end
```

Abbildung 16 Code im Matlab Function-Block

5.2 Bearbeitung der Ein- und Ausgangsvariablen von einem Matlab Function-Block - Property Inspector

Über das Feld „Edit Data“ öffnet sich das Fenster „Symbols“. Hier sind alle Ein- und Ausgangsvariablen aufgelistet. Bei Auswahl einer dieser Größen, öffnet sich ein weiteres Fenster, in dem diese weiter spezifiziert werden kann. Unter der Eigenschaft „Scope“ lässt sich festlegen, ob der Wert ein Eingang, ein Ausgang oder ein Parameter sein soll. Für die Nutzung der Option „Parameter“ muss eine „Mask“ erstellt werden. In dieser ist eine Definition möglich, woher der Parameter bezogen werden soll. Dabei ist sowohl ein händisch festgelegter Wert möglich als auch eine Variable aus dem Matlab Workspace auswählbar. In den hier zu lösenden Aufgaben wird beispielsweise der RoutePlan über diesem Wege dem Simulinkmodell übergeben. Die Mask wird später weiter erläutert.

Unter der Eigenschaft „Size“ ist die Größe der Variable definierbar. Mit dem Wert -1 wird die Größe basierend auf der Eigenschaft „Scope“ bestimmt. Es lässt sich allerdings auch manuell festlegen. Dabei ist ein Skalar (bsp. 1), ein Vektor (bsp. [6 1]) oder eine Matrix (bsp. [3 3]) möglich.

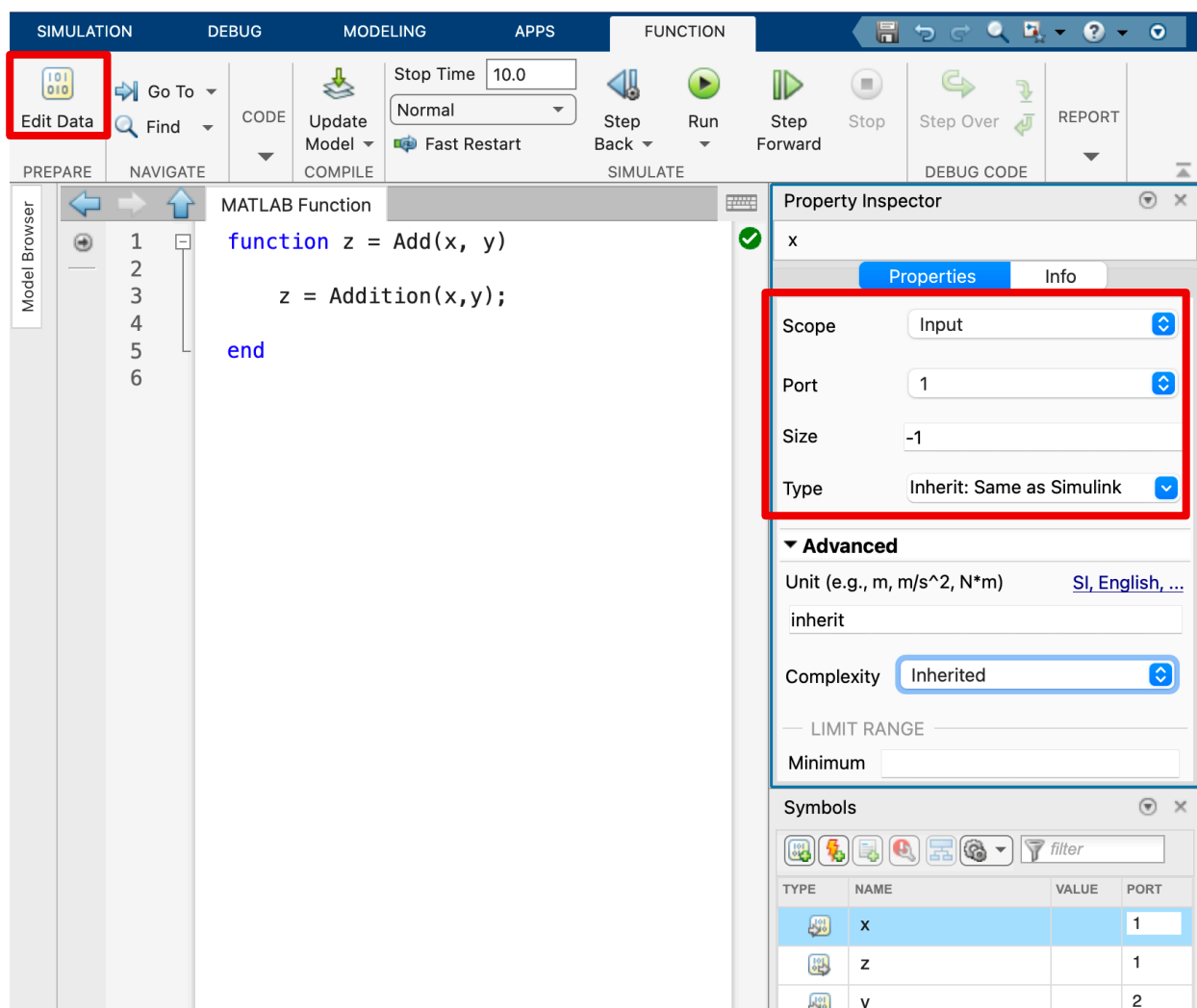


Abbildung 17 Eingabefenster Matlab Function-Block

Ein wichtiges Feld ist zudem unter der Eigenschaft „Type“ zu finden. Hier ist wählbar, welcher Datentyp die Variable angehören soll. Hier ist unter anderem der „Bus: <object name>“ wichtig. Durch die Wahl dieses Datentyps gibt der Matlab Function-Block sofort ein Bus-Signal aus. Dabei kann der Bus ausgewählt werden, der durch die Helper-Datei „helperCreateBus.m“ erstellt wird.

Unter Advanced lässt sich bei Ausgabevariablen die Eigenschaft „Variable size“ auswählen. Diese Eigenschaft ermöglicht, dass eine Variable im Laufe der Simulation ihre Größe ändert. Dies ist beispielsweise im Path Planner bei der Ausgabe der RefPoses notwendig. Hier verändert sich je nach Ausgangspose und Zielpose die Anzahl der RefPosen. Aus diesem Grund muss die Größe variabel sein.

Weiterhin befindet sich hier die Eigenschaft „Tunable“. Diese Option ist nur bei Parametern verfügbar. Parameter mit einem Häkchen bei „Tunable“ können ohne Neukompilierung während einer Simulation verändert werden. Diese Option wird bei den hier vorliegenden Aufgaben jedoch nicht benötigt.

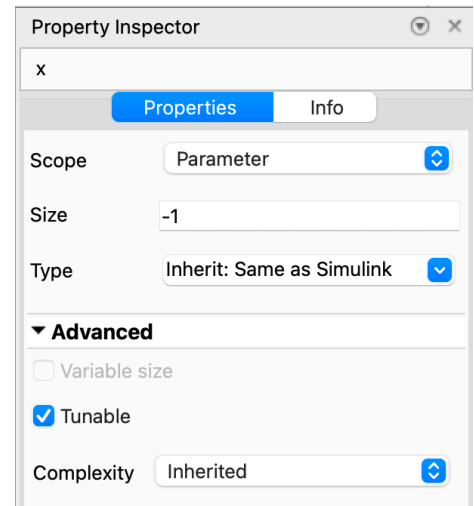


Abbildung 18 Property Inspector bei einem Parameter

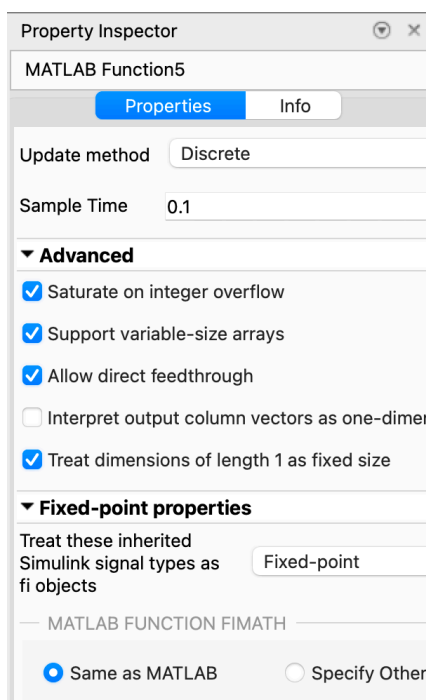


Abbildung 19 Übergeordnete Einstellungen im Property Inspector

Abbildung 19 zeigt noch weitere Einstellungen im Property Inspector. In diesem Anwendungsfall ist die Sample Time am wichtigsten.

5.3 Mask

In diesem Beispiel soll das oben genannte Addition Beispiel genutzt werden, um die Funktionsweise der Mask zu veranschaulichen. Mit der Mask soll aus dem Matlab Workspace die Variable x bezogen werden. Abbildung 20 zeigt wo die Mask implementiert werden kann.

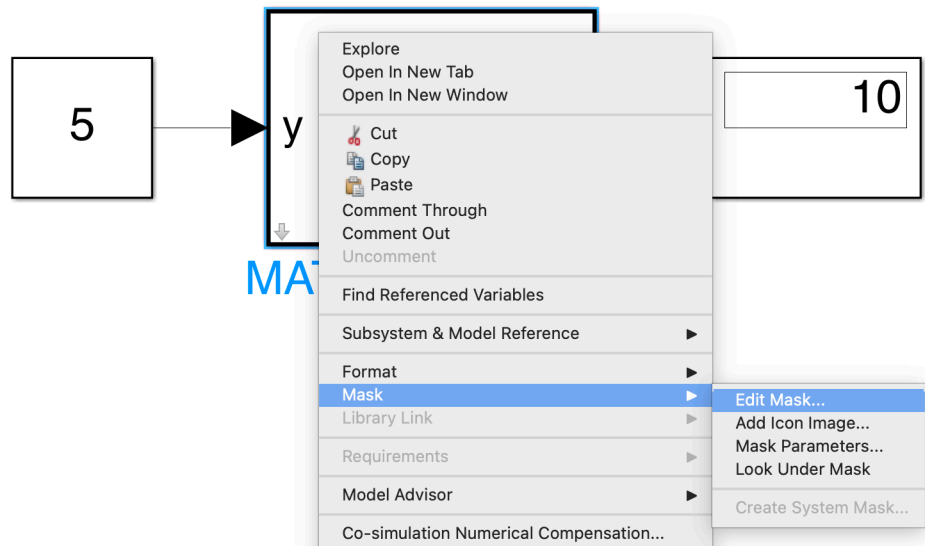


Abbildung 20 Auswahl der Option Mask im Addition Beispiel

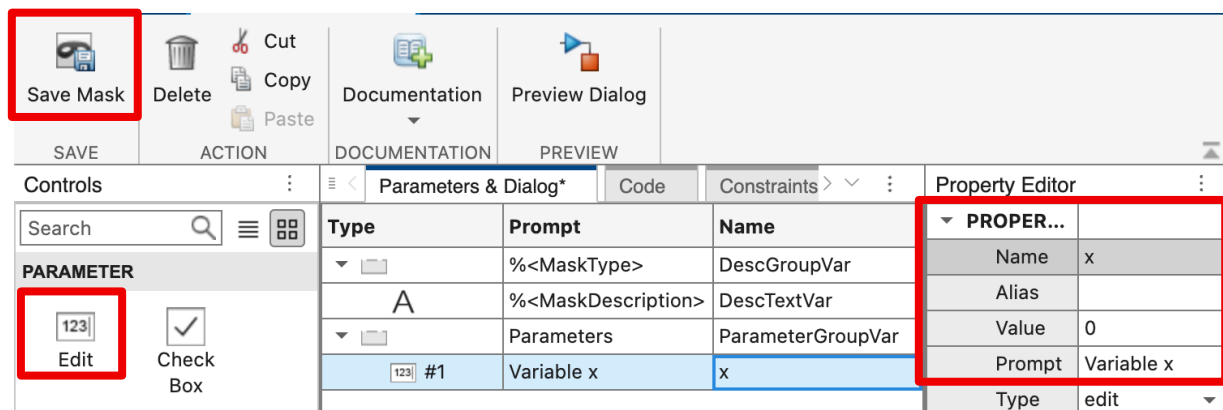


Abbildung 21 Fenster zur Bearbeitung der Mask

In dem auf Abbildung 21 gezeigten Fenster lässt sich über die Option „Edit“ eine neue Variable in die Mask mit aufnehmen. Über „Name“ und „Prompt“ lässt sich diese Variable benennen. Nach Bestätigung unter „Save Mask“ unter dem erneuten Doppelklick auf den Matlab Function-Block kann nun eine Variable für x aus dem Matlab Workspace ausgewählt werden. In Abbildung 22 wird die erstellte Mask mit der Auswahl an Variablen gezeigt.

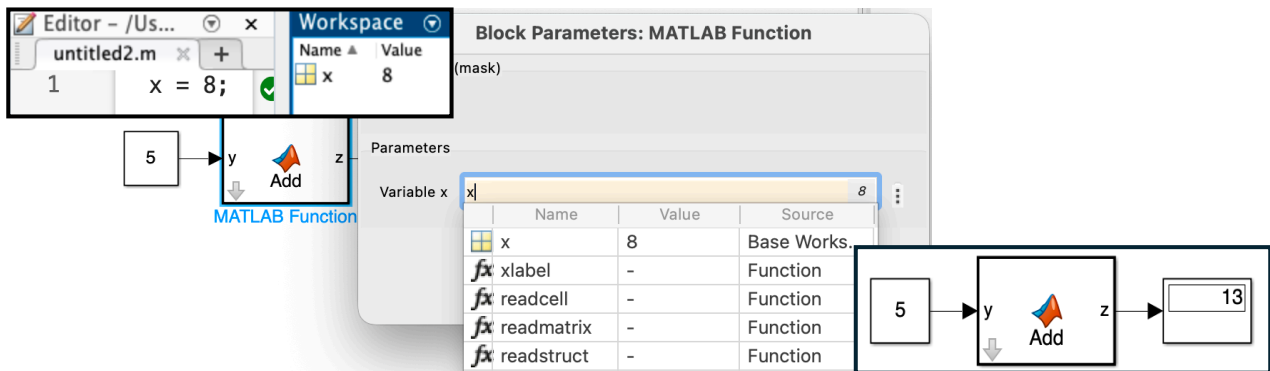


Abbildung 22 Mit der Mask kann die Variable aus dem Matlab Workspace geladen werden

Unter der Kategorie „Code“ auf Abbildung 23 lässt sich zudem eine Callback Function integrieren. Diese Funktion wird später bei der Integration des RoutePlans und der Costmap in das Simulinkmodell wichtig. Die Callback Function wird dabei benutzt, um den RoutePlan in ein Struct umzuwandeln. Bei dem Laden der Costmap wird durch die Callback Function die helper-Datei `helperSLCreateUtilityStruct.m` aufgerufen.

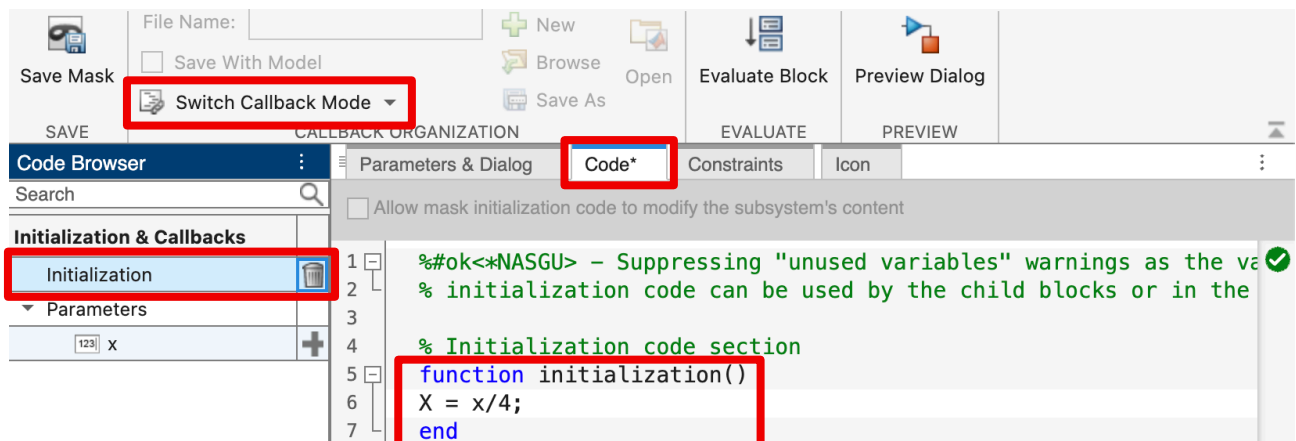


Abbildung 23 Implementierung einer Callback Function

6 Literaturverzeichnis

- [1] Ö. Dönmez, *Entwicklung eines Automated Valet Parking Systems im Rahmen des Forschungsprojekts ANTON*, 1st ed. 2023. Wiesbaden: Springer Fachmedien Wiesbaden, Springer Vieweg, 2023 [Online]. Verfügbar unter: <https://doi.org/10.1007/978-3-658-43117-4>.
- [2] D. W. Simone, J. M. Pavos, N. N. Kidane und M. Avran, *Efficient Path Planning for Differential Drive Mobile Robots Using the Rapidly-Exploring Random Tree (RRT) Algorithm*, 2025 7th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (ICHORA), Ankara, Türkiye, 2025, doi: 10.1109/ICHORA65333.2025.11017146.
- [3] „mathworks.com,“ The MathWorks, Inc, [Online]. Available: <https://de.mathworks.com/>.